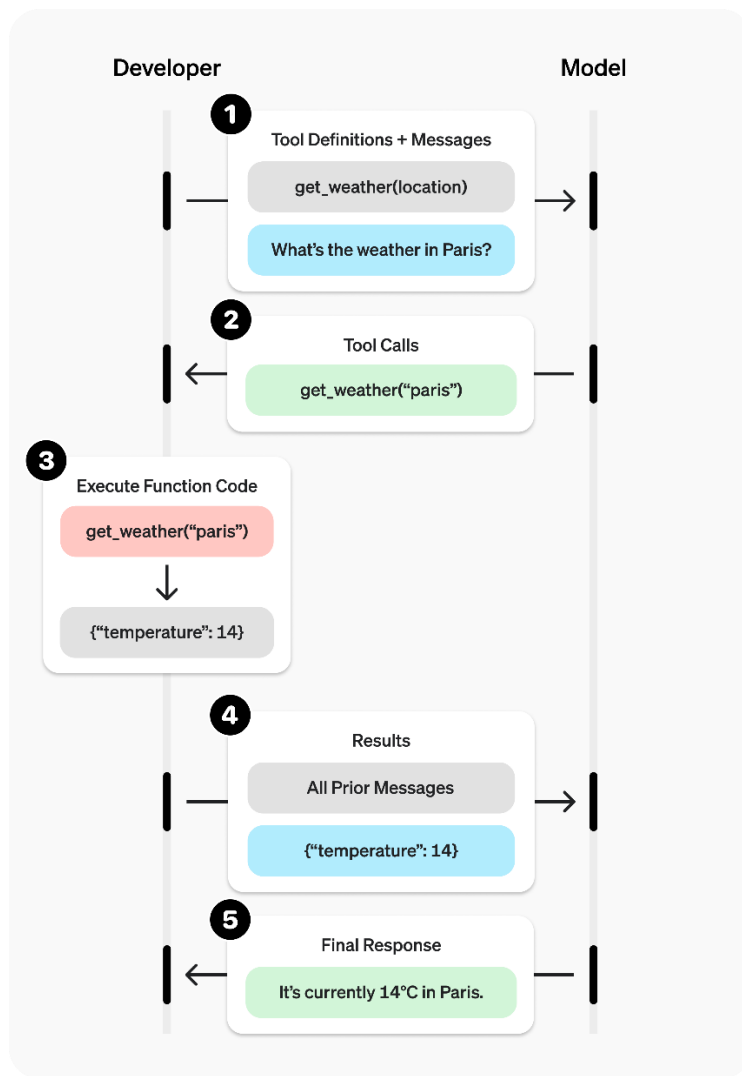


## Using function calling with OpenAI Service

- Function calling is useful when you are building an application that **bridges the models and functionality of your application**. For example, you can give the model access to functions that query a database in order to build an AI assistant that can help users with their orders, or functions that can interact with the UI.
- The latest versions of gpt-3.5-turbo and gpt-4 are fine-tuned to work with functions and are able to both determine when and how a function should be called.
- If one or more functions are included in your request, the model determines if any of the functions should be called based on the context of the prompt.
- When the model determines that a function should be called, it responds with a JSON object including the arguments for the function.

At a high level you can break down working with functions into three steps:

1. Call the chat completions API with **your functions** and **the user query**
2. Use the model's response to call your API or function.
3. Call the chat completions API again, including the response from your function to get a final response.



**Tool choice:** By default the model will determine when and how many tools to use. You can force specific behavior with the **tool\_choice** parameter.

1. **Auto:** (Default) Call zero, one, or multiple functions. **tool\_choice: "auto"**
2. **Required:** Call one or more functions. **tool\_choice: "required"**
3. **Forced Function:** Call exactly one specific function. **tool\_choice: {"type": "function", "function": {"name": "get\_weather"}}**
4. **None:** Imitate the behavior of passing no functions.

**Note:** If the model decides that no function should be called, then the response will contain a direct reply to the user as a regular chat completion response.

**Strict mode:** Setting strict to true will ensure function calls reliably adhere to the function schema, instead of being best effort. We recommend always enabling strict mode.

Notes: <https://platform.openai.com/docs/guides/function-calling?example=get-weather&strict-mode=disabled#additional-configurations>

### Single tool/function calling example

```
import requests
import json
from openai import OpenAI
import os
from dotenv import load_dotenv

# Following is for Intuit only
import truststore
truststore.inject_into_ssl()

load_dotenv()

client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

#*****Step1: Create a function to get the weather*****
# The function will take latitude and longitude as input and return the current temperature in celsius.
def get_weather(latitude, longitude):
    response = requests.get(f"https://api.open-
meteo.com/v1/forecast?latitude={latitude}&longitude={longitude}&current=temperature_2m,wind_speed_1
0m&hourly=temperature_2m,relative_humidity_2m,wind_speed_10m")
    data = response.json()
    return data['current']['temperature_2m']

#*****Step2: Call model with functions defined – along with your system and user messages.*****
get_weather_schema = {
    "type": "function",
    "name": "get_weather",
    "description": "Get current temperature for provided coordinates in celsius.",
    "parameters": {
        "type": "object",
        "properties": {
```

```

        "latitude": {"type": "number"},
        "longitude": {"type": "number"}
    },
    "required": ["latitude", "longitude"],
    "additionalProperties": False
},
"strict": True
}

```

```
tools = [get_weather_schema]
```

```
input_messages = [{"role": "user", "content": "What's the best clothing based on weather in Paris today?"}]
```

```
response = client.responses.create(
```

```
    model="gpt-6-luna",
```

```
    input=input_messages,
```

```
    tools=tools
```

```
)
```

```
#input_messages += response.output # append model's function call item
```

```
*****Step3: Model decides to call function(s) – model returns the name and input arguments.*****
```

```
# Extract the arguments from the function
```

```
tool_call = next(item for item in response.output if item.type == "function_call")
```

```
args = json.loads(tool_call.arguments)
```

```
print(tool_call.name)
```

```
print(args)
```

```
*****Step 4: Execute function code – parse the model's response and handle function calls.*****
```

```
result = get_weather(args["latitude"], args["longitude"])
```

```
*****Step 5: Supply model with results – so it can incorporate them into its final response.*****
```

```
input_messages = [{ # append result message
```

```
    "type": "function_call_output",
```

```
    "call_id": tool_call.call_id,
```

```
    "output": str(result)
```

```
}]
```

```
response_2 = client.responses.create(
    previous_response_id=response.id,
    model="gpt-6-luna",
    input=input_messages,
    tools=tools,
)

print(response_2.output_text)
```

### Example2: Calling Same function multiple times

```
import requests
import json
from openai import OpenAI
import os
from dotenv import load_dotenv

load_dotenv()
client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

def get_weather(latitude, longitude):
    response = requests.get(
        f"https://api.open-
meteo.com/v1/forecast?latitude={latitude}&longitude={longitude}&current=temperature_2m,wind_speed_1
0m&hourly=temperature_2m,relative_humidity_2m,wind_speed_10m"
    )
    data = response.json()
    return data["current"]["temperature_2m"]

get_weather_schema = {
    "name": "get_weather",
    "description": "Get current temperature for provided coordinates in celsius.",
    "parameters": {
        "type": "object",
        "properties": {"latitude": {"type": "number"}, "longitude": {"type": "number"}},
    },
}
```

```

        "required": ["latitude", "longitude"],
        "additionalProperties": False,
    },
    "strict": True,
}

tools = [{"type": "function", **get_weather_schema}]

inputs = [
    {
        "role": "user",
        "content": "What's the best clothing based on weather in Paris and Mumbai today?",
    }
]

previous_response_id = None
while True:
    response = client.responses.create(
        model="gpt-5.6-luna",
        input=inputs,
        previous_response_id=previous_response_id,
        tools=tools,
    )
    previous_response_id = response.id
    if response.output_text:
        print(response.output_text)
        break

    inputs = []
    for item in response.output:
        if item.type == "function_call":
            args = json.loads(item.arguments)
            print(item)
            print(args)
            if item.name == "get_weather":

```

```

        result = get_weather(args["latitude"], args["longitude"])
    inputs.append(
        {
            "type": "function_call_output",
            "call_id": item.call_id,
            "output": str(result),
        }
    )
)

```

## Multiple functions calling

### Example with Weather and News

```

import requests
import json
from openai import OpenAI
import os
from dotenv import load_dotenv

load_dotenv()
client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

def get_weather(latitude, longitude):
    response = requests.get(
        f"https://api.open-
meteo.com/v1/forecast?latitude={latitude}&longitude={longitude}&current=temperature_2m,wind_speed_1
0m&hourly=temperature_2m,relative_humidity_2m,wind_speed_10m"
    )
    data = response.json()
    return data["current"]["temperature_2m"]

get_weather_schema = {
    "name": "get_weather",
    "description": "Get current temperature for provided coordinates in celsius.",
    "parameters": {
        "type": "object",
        "properties": {"latitude": {"type": "number"}, "longitude": {"type": "number"}},
    }
}

```

```

    "required": ["latitude", "longitude"],
    "additionalProperties": False,
},
"strict": True,
}

# Define the `get_news` function to retrieve news articles based on a given topic
news_api_key = "c86245eca72141828e351e1ea1e92fae"
def get_news(topic):
    print(f"In get news about {topic}")
    url = (
        f"https://newsapi.org/v2/everything?q={topic}&apiKey={news_api_key}&pageSize=5"
    )
    try:
        response = requests.get(url)
        if response.status_code == 200:
            news = json.dumps(response.json(), indent=4)
            news_json = json.loads(news)

            # Access all the fields == loop through
            status = news_json["status"]
            total_results = news_json["totalResults"]
            articles = news_json["articles"]
            final_news = []

            # Loop through articles
            for article in articles:
                source_name = article["source"]["name"]
                author = article["author"]
                title = article["title"]
                description = article["description"]
                url = article["url"]
                content = article["content"]
                title_description = f"""
                    Title: {title},

```

```

        Author: {author},
        Source: {source_name},
        Description: {description},
        URL: {url},
        Content: {content}
    """

    final_news.append(title_description)

    return final_news

else:

    return []

except requests.exceptions.RequestException as e:

    return f"Error occurred during API Request: {e}"

get_news_schema = {
    "name": "get_news",
    "description": "Get news articles based on a given topic.",
    "parameters": {
        "type": "object",
        "properties": {
            "topic": {"type": "string"}
        },
        "required": ["topic"],
        "additionalProperties": False
    },
    "strict": True
}

tools = [{"type": "function", **get_weather_schema}, {"type": "function", **get_news_schema}]

inputs = [
    {
        "role": "user",
        "content": "What's the best clothing based on weather in Paris and Mumbai today? Also let me know the latest news about these cities.",
    }
]

```

```

]

previous_response_id = None
while True:
    response = client.responses.create(
        model="gpt-5.6-luna",
        input=inputs,
        previous_response_id=previous_response_id,
        tools=tools,
    )
    previous_response_id = response.id
    if response.output_text:
        print(response.output_text)
        break

inputs = []
for item in response.output:
    if item.type == "function_call":
        args = json.loads(item.arguments)
        if item.name == "get_weather":
            result = get_weather(args["latitude"], args["longitude"])
        elif item.name == "get_news":
            result = get_news(args["topic"])
        inputs.append(
            {
                "type": "function_call_output",
                "call_id": item.call_id,
                "output": str(result),
            }
        )

```